

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

ISSUE LOCALIZATION

Data Science Project — Progress Report

Người thực hiện:

Cao Van Dong
Luong Ngoc Vu Long
Pham Thanh Phong
Nguyen Ngoc Minh Hieu
Trinh Luong Viet

ONE LOVE. ONE FUTURE.

Problem Statement

User Need

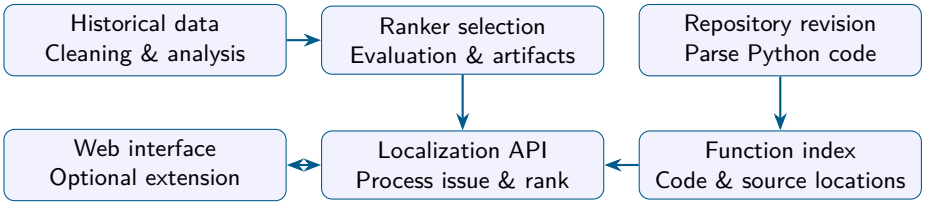
- **Users**: developers and contributors investigating issues in Python repositories.
- **Pain point**: issue reports describe symptoms, but developers must search the code to find where to investigate.
- **Goal**: help users find relevant existing functions and reduce manual code search.
- **Deliverable**: a web application, with an extension as a possible integration.

Data Science Problem

- **Input**: an issue URL or description and a selected repository revision.
- **Output**: ranked functions with file paths, line numbers, code previews and source links.
- **Learning problem**: supervised learning to rank existing functions by their relevance to an issue, using historical issue–function examples with patch-derived labels.

Proposed Application & System Design

User workflow: select a repository, submit an issue, inspect suggestions, open code.



- **Results**: show ranked functions with code previews and links to the selected revision.
- **Indexing**: prepare code once per revision and reuse it for subsequent issue queries.
- **Status**: application design is proposed. Current work focuses on the data pipeline.

Data Collection: Selecting Repositories

Step 1 — Top PyPI Packages

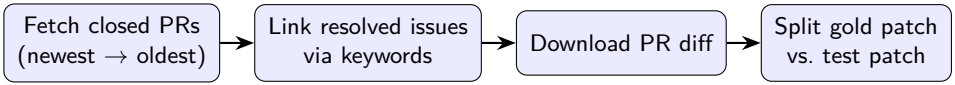
- Source: public top-PyPI dump (hugovk.dev), ranked by download counts.
- Started with **11,000** packages and collected metadata, including source links.

Step 2 — GitHub Repository Candidates

- Extract `github.com/owner/repo` links from package metadata.
- Clean, deduplicate, normalize $\rightarrow$ **7,773** unique repositories for mining.

Data Collection: Mining Pull Requests & Linking Issues

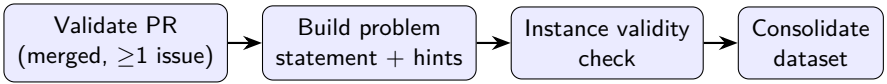
Step 1 | Step 2 | **Step 3** | Step 4 | Step 5 | Step 6



- **PR fetching**: GitHub API, up to 3,000 PRs per repository; automatic rate-limit backoff.
- **Issue linking**: search PR titles, descriptions and commit messages for resolving references such as *fixes #123* or *closes #123*.
- **Diff handling**: use `unidiff` to separate test changes from the production patch.
- **Output**: **4.69M** PRs collected across the **7,773**-repository mining list.

Data Collection: Building Task Instances

Step 1 | Step 2 | Step 3 | **Step 4** | Step 5 | Step 6



Instance Fields

- **Issue text**: linked issue title and body.
- **Optional hints**: issue comments from before the first PR commit.
- **Traceability**: repository, base commit, patches and instance ID.

Filters & Output

- Keep **merged** PRs with a linked issue, non-empty patch and issue text.
- Group by repository, shuffle within each repo and remove test patches.
- Output: **396K** issue–patch tasks.

Data Processing: Function Labels & Negatives

Step 5 — Function-Level Labeling

- Reconstruct the code at the base commit and extract Python functions and methods.
- Map patch changes to **existing functions** to obtain positive (issue, function) examples.
- Retain code and source paths for ranking and navigation in the planned app.

Step 6 — Negative Mining & Filtering

- Score candidate functions with a pretrained code encoder and select challenging negatives from the same repository.
- Apply rank-based filtering to positive examples and candidate negatives.
- Export issue–function candidate lists for ranking experiments.

Collected Data Overview

Key Statistics

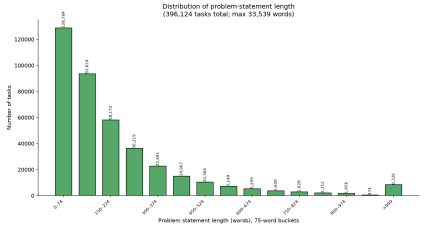
Item	Count
Initial PyPI packages	**11,000**
GitHub repository candidates	**7,773**
Pull requests collected	**4.69M**
Task instances (issues + patches)	**396K**

Instance Schema

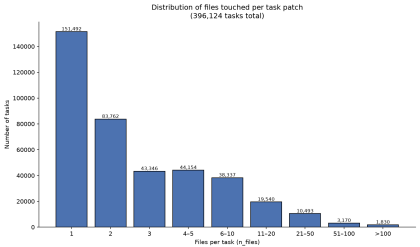
```
{
  "instance_id": ...,
  "repo": ...,
  "base_commit": ...,
  "issue_numbers": [...],
  "patch": ...,
  "problem_statement": ...,
  "hints_text": ...,
  "created_at": ...
}
```

Data Distributions & Quality

Issue / Problem Statement Length



Patch Breadth (files per task)



- **Issue length**: about 56% have fewer than 150 words, motivating analysis of missing context.
- **Patch breadth**: about 59% touch one or two files, but broader tasks can have multiple targets.
- **Next checks**: audit label relevance and duplicates, and inspect parsing failures.

Evaluation Plan

Ranking Quality & Data Analysis

- **Comparisons**: BM25 keyword search, pretrained code embeddings and optional fine-tuning.
- **Measures**: Recall@5 / Recall@10 for target coverage and MRR for the first relevant result.
- **Data experiments**: compare filtering choices and performance across issue lengths and patch sizes.
- **Validation**: separate repositories across training, validation and test sets.

Application Usefulness

- **Speed**: measure initial indexing time and query response time separately.
- **User task**: compare time to find relevant code with ordinary repository search.
- **Usability**: test issue submission, result inspection and navigation to source code.
- **Feedback**: collect relevance judgments and review failure cases for later improvements.

Summary & Next Steps

Current Progress

- Implemented collection and processing scripts for linked issues, patches and function candidates.
- Reported collection: **396K** task instances, with **7,773** repository candidates in the mining list.
- Initial EDA describes issue lengths and patch breadth. Label auditing and final filtering remain.

Next Steps & Final Deliverable

- **Data**: finalize filters and labels, including new/deleted functions and addition-only edits.
- **Ranking**: compare baselines and select a method using held-out repositories.
- **Application**: build the indexing service, localization API and web interface. Explore an extension integration.
- **Delivery**: demonstrate issue submission, ranked code results and source navigation, then evaluate with users.



THANK YOU !